Laboratorio Calificado 01

2026-1

ÍNDICE

1 Instrucciones 3
2 Contexto 3
3 Requisitos del Problema 3
3.1 Datos del Sistema: 3
3.2 Funcionalidad: 3
3.3 Interacción con el Usuario: 4
4 Tareas para Resolver 4
5 Entregable 5
6 Código base para comenzar 5
7 Caso de Prueba 7
7.1 Condición inicial: 7
7.2 Entrada: 7
7.3 Salida esperada: 7
8 Rúbrica de Evaluación 8

Laboratorio Calificado 01
Control de Sensores de Temperatura en una Línea de Producción

1 Instrucciones

Se ha creado la tarea "Laboratorio Calificado 1" en la sección Laboratorio Calificado 1 de la página del curso-horario en PAIDEIA. Cada alumno deberá entregar un archivo comprimido (formato ZIP) con todos los archivos del desarrollo realizado en el laboratorio. El nombre del archivo comprimido deberá tener el formato:
L1_<Código del alumno de 8 dígitos>.<Extensión del archivo>
Como ejemplo, el nombre del archivo de alumno 20236969 del horario 06M1 sería "L1_20236969.zip". Es COMPLETA RESPONSABILIDAD DE CADA ALUMNO el colocar nombres correctos a sus archivos y evitar así confusiones al momento de la calificación.

2 Contexto

En un brazo robótico industrial de **10 ejes (servomotores)**, es vital monitorear el consumo de corriente (en **Amperios**) de cada motor. Un consumo muy bajo indica que el motor no tiene carga, mientras que un consumo elevado sugiere que el brazo está cargando un objeto demasiado pesado o que hay una obstrucción mecánica.
El alumno deberá gestionar un arreglo de **10 posiciones** (una por motor) para realizar lo siguiente:

- **Registro de Consumo:** Ingresar la corriente actual de cada uno de los 10 motores.
- **Análisis de Esfuerzo:** Calcular el **consumo promedio** de todo el brazo y determinar cuál es el motor que está realizando el **mayor esfuerzo** (corriente máxima).
- **Detección de Bloqueo (Alerta):** Si un motor supera los **4.5 A**, el sistema debe lanzar una alerta de **"SOBRECARGA DETECTADA"**, ya que esto podría quemar el bobinado del motor.
- **Mantenimiento:** Permitir actualizar el valor de corriente de un motor específico después de liberar una obstrucción.

.

3 Requisitos del Problema

3.1 Datos del Sistema:

Para la implementación del software de monitoreo de carga, se deben considerar las siguientes restricciones:

- **Configuración del Brazo Robótico:** El sistema debe gestionar exactamente **10 servomotores (ejes)** que componen la estructura cinemática del robot.
- **Estructura de Almacenamiento:** Los valores de consumo de corriente deben ser procesados mediante un **arreglo unidimensional de 10 números de punto flotante (float)**.
    - *Nota:* El uso de arreglos es obligatorio para evaluar la gestión de memoria estática y el paso de parámetros por referencia en C++/CLI.
- **Parámetro de Seguridad (Torque Máximo):** Se establece un **umbral de corriente crítica de 4.5 A**. Cualquier valor que iguale o supere esta cifra debe ser tratado como una condición de sobrecarga o bloqueo mecánico, activando una alarma en el sistema.

3.2 Funcionalidad:

El programa debe implementar la lógica de control para un brazo robótico de 10 ejes mediante las siguientes funciones en **C++/CLI**:

1. **void registrarCorrientes(array<float>^ motores)**
    - **Propósito:** Capturar las lecturas de amperaje de los 10 motores.
    - **Lógica:** Solicita al usuario el ingreso de datos (en Amperios) para llenar el arreglo.

2. **void mostrarEstadoMotores(array<float>^ motores)**
   - **Propósito:** Reporte de telemetría actual.
   - **Lógica:** Imprime el consumo de cada motor (ej. "Motor 1: 1.5 A").
3. **float calcularConsumoPromedio(array<float>^ motores)**
   - **Propósito:** Análisis de carga general del brazo.
   - **Lógica:** Retorna el promedio aritmético del amperaje de todos los motores.
4. **float obtenerEsfuerzoMaximo(array<float>^ motores)**
   - **Propósito:** Identificar el motor con mayor carga de trabajo.
   - **Lógica:** Algoritmo para encontrar el valor máximo de corriente en el arreglo.
5. **float obtenerEsfuerzoMinimo(array<float>^ motores)**
   - **Propósito:** Identificar motores sin carga o desconectados.
   - **Lógica:** Algoritmo para encontrar el valor mínimo de corriente.
6. **void verificarSobrecargas(array<float>^ motores)**
   - **Propósito:** Seguridad industrial.
   - **Lógica:** Evalúa si algún motor supera el **umbral crítico de 4.5 A**. Si es así, debe mostrar un mensaje de alerta indicando el riesgo de daño por sobrecarga.
7. **void recalibrarMotor(array<float>^ motores, int idMotor, float nuevaCorriente)**
   - **Propósito:** Mantenimiento y ajuste.
   - **Lógica:** Permite actualizar el valor de un motor específico tras liberar una obstrucción o realizar un cambio de pieza.

3.3 Interacción con el Usuario:

El programa principal (main) debe actuar como la unidad de control del sistema, desplegando un menú interactivo que permita al usuario gestionar los 10 ejes del robot. El sistema debe ejecutarse dentro de un bucle iterativo (do-while o while) hasta que el usuario decida finalizar la sesión.

Opciones obligatorias del menú:

- Registrar corrientes iniciales: Captura el amperaje de los 10 motores.
- Mostrar estado de la red: Despliega el consumo actual de todos los ejes.
- Calcular consumo promedio: Muestra la carga media del brazo robótico.
- Identificar esfuerzos extremos: Muestra cuál es el motor con mayor y menor consumo.
- Verificar sobrecargas críticas: Ejecuta la lógica de seguridad (Alerta si $\ge$ 4.5 A).
- Recalibrar motor específico: Permite actualizar el dato de un solo motor por su ID.
- (Q) Finalizar sistema: Sale del bucle y cierra el programa.

- Se deben emplear **estructuras de control selectivas** (switch-case o if-else) para dirigir las opciones del menú.
- El uso de **estructuras iterativas** (for o while) es mandatorio para el procesamiento del arreglo dentro de las funciones.
- El programa debe ser amigable: tras ejecutar una opción, debe permitir al usuario ver el resultado antes de volver a mostrar el menú.

4 Tareas para Resolver

El estudiante deberá completar las siguientes actividades para garantizar el funcionamiento del sistema de monitoreo:

1. Definición de la Estructura: Declarar e inicializar un arreglo de 10 elementos de tipo punto flotante (float) que representará la carga de corriente de los 10 ejes del brazo robótico.
2. Implementación Modular: Desarrollar la lógica interna de cada una de las 7 funciones solicitadas en la sección 3.2, asegurando el correcto paso del arreglo por referencia (^).

3. Interfaz de Control: * Consola: Implementar un menú basado en switch-case dentro de un bucle que responda a las opciones (1-6 y Q).
    - GUI (Opcional): Si se opta por una interfaz visual, cada función debe estar vinculada a un evento de botón, mostrando los resultados en etiquetas (labels) o cuadros de lista (listbox).
4. Lógica de Validación (Puntos Críticos):
    - Protección de Datos: No permitir el ingreso de valores de corriente negativos (físicamente imposibles en este contexto).
    - Control de Índices: Validar que al recalibrar un motor, el identificador ingresado esté estrictamente en el rango 1-10.

5 Entregable

Cree un nuevo proyecto en C++/CLI (consola o GUI) que contenga un programa completo que:

- Implemente correctamente las funciones descritas.
- Utilice tipos de datos adecuados, estructuras selectivas, iterativas y arreglos.
- Interactúe con el usuario a través de un menú sea por consola o por GUIs.
- Cumpla con las validaciones y requisitos del contexto.

6 Código base para comenzar

C++/CLI

```
#include "pch.h"

using namespace System;

// --- Prototipos de Funciones (El alumno debe implementar la lógica) ---

void registrarCorrientes(array<float>^ motores) {
    Console::WriteLine("\n[TAREA] Implementar bucle para ingreso de 10
lecturas...");
    // TODO: Solicitar datos al usuario y llenar el arreglo 'motores'
}

void mostrarEstadoMotores(array<float>^ motores) {
    Console::WriteLine("\n--- Estado Actual de los Ejes ---");
    // TODO: Recorrer el arreglo y mostrar Motor e ID
}

float calcularConsumoPromedio(array<float>^ motores) {
    float suma = 0;
    // TODO: Calcular y retornar el promedio
    return 0.0f;
}

float obtenerEsfuerzoMaximo(array<float>^ motores) {
    // TODO: Encontrar y retornar el valor más alto del arreglo
    return 0.0f;
}

void verificarSobrecargas(array<float>^ motores) {
```

```
    float umbral = 4.5f;
    Console::WriteLine("\n--- Verificando Seguridad del Brazo ---");
    // TODO: Si motor[i] >= 4.5, mostrar alerta de sobrecarga
}

void recalibrarMotor(array<float>^ motores, int idMotor, float nuevaCorriente) {
    // TODO: Validar idMotor (1-10) y actualizar la posición idMotor-1
}

// --- Función Principal ---

int main(array<System::String ^> ^args) {
    // Definición del arreglo de 10 servomotores
    array<float>^ motores = gcnew array<float>(10);
    String^ opcion;

    do {
        Console::WriteLine("\n========================================");
        Console::WriteLine("  SISTEMA DE CONTROL DE TORQUE v1.0");
        Console::WriteLine("========================================");
        Console::WriteLine("1. Registrar corrientes iniciales");
        Console::WriteLine("2. Mostrar estado de la red (10 ejes)");
        Console::WriteLine("3. Calcular consumo promedio total");
        Console::WriteLine("4. Identificar esfuerzos extremos (Max)");
        Console::WriteLine("5. Verificar sobrecargas críticas (>4.5A)");
        Console::WriteLine("6. Recalibrar motor específico");
        Console::WriteLine("Q. Finalizar sistema");
        Console::Write("\nSeleccione una opcion: ");

        opcion = Console::ReadLine()->ToUpper();

        if (opcion == "1") {
            registrarCorrientes(motores);
        }
        else if (opcion == "2") {
            mostrarEstadoMotores(motores);
        }
        else if (opcion == "3") {
            float prom = calcularConsumoPromedio(motores);
            Console::WriteLine("El consumo promedio es: {0} A", prom);
        }
        else if (opcion == "4") {
            float max = obtenerEsfuerzoMaximo(motores);
            Console::WriteLine("El esfuerzo máximo detectado es: {0} A", max);
        }
        else if (opcion == "5") {
            verificarSobrecargas(motores);
        }
        else if (opcion == "6") {
            Console::Write("Ingrese ID del motor (1-10): ");
            int id = Int32::Parse(Console::ReadLine());
```

```
            Console::Write("Ingrese nueva lectura de corriente: ");
            float nCorriente = float::Parse(Console::ReadLine());
            recalibrarMotor(motores, id, nCorriente);
        }

    } while (opcion != "Q");

    Console::WriteLine("\nSistema cerrado correctamente.");
    return 0;
}
```

7 Caso de Prueba

7.1 Condición inicial:

El sistema inicia con el arreglo de 10 motores en cero (0.0 A).

7.2 Entrada:

- Registro de corrientes: 1.5, 2.2, 3.0, 4.8, 2.5, 3.8, 1.2, 5.5, 1.9, 3.2
- Mostrar estado de todos los motores.
- Consultar promedio de carga.
- Consultar máxima y mínima carga detectada.
- Verificar sobrecargas (Umbral≥ 4.5A).
- Recalibrar Motor 4 a un nuevo valor de 2.8 A.

7.3 Salida esperada:

- ```
  Seleccione una opción: 1
     Ingrese corriente para Motor 1: 1.5
     Ingrese corriente para Motor 2: 2.2
     Ingrese corriente para Motor 3: 3.0
     Ingrese corriente para Motor 4: 4.8
     Ingrese corriente para Motor 5: 2.5
     Ingrese corriente para Motor 6: 3.8
     Ingrese corriente para Motor 7: 1.2
     Ingrese corriente para Motor 8: 5.5
     Ingrese corriente para Motor 9: 1.9
     Ingrese corriente para Motor 10: 3.2
  ```

- ```
  Seleccione una opción: 2
  Estado actual de los ejes:
     Motor 1: 1.5 A
     Motor 2: 2.2 A
     Motor 3: 3.0 A
     Motor 4: 4.8 A
     Motor 5: 2.5 A
     Motor 6: 3.8 A
     Motor 7: 1.2 A
     Motor 8: 5.5 A
     Motor 9: 1.9 A
     Motor 10: 3.2 A
  ```

- **Seleccione una opción: 3**

    - Consumo promedio del brazo: **2.96 A**
- **Seleccione una opción: 4**
    - Esfuerzo máximo detectado: **5.5 A**
    - Esfuerzo mínimo detectado: **1.2 A**
- **Seleccione una opción: 5**
    - **[ALERTA]** Motor 4 supera el umbral crítico con 4.8 A.
    - **[ALERTA]** Motor 8 supera el umbral crítico con 5.5 A.
- **Seleccione una opción: 6**
    - Ingrese ID del motor a recalibrar (1-10): **4**
    - Ingrese nueva lectura de corriente: **2.8**
    - *Sistema: Motor 4 actualizado correctamente a 2.8 A.*

Resultado esperado después de la actualización:

- Si se vuelve a calcular el promedio, este debe disminuir.
- Si se verifica sobrecargas (Opción 5), el **Motor 4 ya no debe aparecer**, pero el **Motor 8 sí**.

En el caso de desarrollar un programa basado en GUIs deberá crear el formulario con los controles visuales necesarios para la interacción con el usuario y la invocación a cada una de las opciones.

8 Rúbrica de Evaluación

Puntaje Máximo: 20 puntos

Criterio	Excelente	Bueno	Regular	Deficiente	No Cumple
Uso de Arreglos (4 pts)	Declara y usa correctamente el arreglo para todas las operaciones	Declara y usa con leves errores	Uso parcial o limitado	Uso incorrecto	No usa arreglos
Definición y Uso de Funciones (5 pts)	Todas las funciones implementadas correctamente	Implementa la mayoría con leves errores	Implementa parcialmente	Implementa mínimamente	No usa funciones
Estructuras de Control (4 pts)	Aplica selectivas e iterativas de manera correcta y eficiente	Uso adecuado con pequeños errores	Uso parcial o incompleto	Uso incorrecto	No aplica
Validaciones (3 pts)	Valida entradas y límites correctamente	Valida parcialmente	Valida de forma incompleta	Validaciones mínimas	No valida
Funcionalidad Global (4 pts)	El programa cumple todos los requisitos y funciona sin errores	Funciona con leves errores	Funciona parcialmente	Funcionalidad mínima	No funciona